

VoxGuard

A Real-Time Voice Trust Layer

The problem in plain words

A finance employee gets a call from their CFO. The voice sounds exactly right. Same tone, same accent, same little pauses the CFO always makes. The CFO asks for an urgent wire transfer before a deal closes. The employee sends it.

The CFO never made that call. It was a clone, built from a few seconds of audio scraped off a company town hall video on YouTube.

This is not a hypothetical anymore. Voice cloning has gone from a research demo to something anyone can do in minutes, and criminals are using it to impersonate CXOs, government officials, and trusted colleagues over ordinary phone and VoIP calls. The old ways of checking who is on the other end of a call, recognising a voice, checking the caller ID, calling back the number, do not work when the voice itself is fake.

So right now, banks, enterprises, and government offices are exposed on every single voice channel they use, and they have no automated way to know it.

Why nobody has solved this yet

The obvious answer sounds simple. Run every call through a deepfake detector, get a yes or no, done.

It is not that simple, for three reasons.

Real time is hard. A detector that tells you a call was fake five minutes after the money is gone is useless. The system has to listen to a live audio stream and produce a risk score while the conversation is still happening, fast enough that someone can act on it before they approve anything.

One signal is not enough. A cloned voice can pass a simple “does this sound human” check quite easily. You need multiple independent ways of catching it: the raw acoustic fingerprint of how the audio was generated, the rhythm and micro-timing of natural speech that generators still get slightly wrong, and whether this voice matches what we have heard from this same person before. Any one of these alone gives false confidence.

A number is not an answer. Telling a bank teller “73% risk” is not useful on its own. Someone needs to see that score change live, understand roughly why it is high, and get a clear recommendation, like asking for a callback or a second factor, before they approve a transfer.

So the real problem is not “can we detect a cloned voice”. It is “can we build a live trust layer that sits on top of a call, watches it the whole way through, and tells a human what to do about it, in time to matter”.

What we are building

VoxGuard is a real-time voice authenticity layer that sits between an incoming call and the person receiving it.

The part people see is simple. A call comes in. A risk meter appears on screen and starts moving as the person talks. If the voice looks synthetic or cloned, the meter climbs, a plain-English reason appears next to it, such as “unnatural pitch micro-variation detected”, and the system recommends a next step, like a callback to a known number or an extra approval step, before anything sensitive happens.

The part that actually matters is underneath. Every few seconds of audio gets pulled apart three different ways at once, the results get combined into one live score, and that score gets compared against what has been heard from this caller before, so the system gets more confident the longer a genuine relationship exists between two people.

The name

A voice is supposed to be proof that you are talking to a real person. VoxGuard exists to check that proof, continuously, for as long as the call lasts. It is a guard standing at the door of every voice channel a bank, enterprise, or government office relies on.

How it actually works, step by step

Say a call comes into a bank's contact centre, and the caller claims to be a high-value client authorising a transfer.

1. The call is picked up and streamed in near real time, a few seconds of audio at a time, instead of waiting for the whole call to finish.
2. Each chunk goes through acoustic and spectral analysis, looking for the specific artefacts that voice-generation models leave behind, things a human ear cannot pick up but a model trained on both real and synthetic speech can.
3. In parallel, the same chunk goes through a prosody and rhythm check, looking at pitch contours, pauses, and the tiny timing variations that are very hard for a generator to fake convincingly over a full conversation.
4. If this caller has genuine samples on file, the system runs a consistency check against them, and flags any drift from what this person actually sounds like.
5. All three signals are combined into one running risk score, updated continuously, not just once at the end of the call.
6. If the score crosses a threshold, an alert fires straight away, with the reason shown in plain language, and a recommended action, such as a callback or a supervisor escalation.
7. Everything is logged, minimally and privacy-consciously, so a security team can review exactly what was flagged and why, without a full recording sitting in central storage by default.

The whole time, the person on the receiving end sees one thing clearly: a live number, going up or down, with a reason attached.

Why this is genuinely AWS native

If we had just built “a python script that scores an audio file after the fact”, there would be very little AWS in it, and nothing about it would need to be live. That would be a weak project, because the entire point of this problem is that the detection has to happen while the call is still going on.

VoxGuard is different because the thing that makes it live, and the thing that makes it usable inside a bank or enterprise, both sit on real AWS building blocks, not just AWS hosting.

Amazon Chime SDK is doing the real-time streaming. This is the piece most teams would get wrong by trying to hand-roll. Live audio has to be captured, chunked, and delivered to a detection pipeline with low enough latency that a risk score is still useful, and it has to work the same way whether the call is coming over telephony, VoIP, or an enterprise collaboration platform. Chime SDK is built by AWS specifically for this problem. Take it out, and you are building call infrastructure from scratch instead of building the actual detection system, which is the wrong use of four days.

Amazon Bedrock is doing the translation from score to explanation. A risk number by itself does not tell a bank teller what to do. Bedrock turns “0.82 synthetic probability” into “flagged: unnatural pitch micro-variation detected, recommend callback verification”, which is exactly the kind of plain-language, actionable output the problem statement asks for, and doing this well with hand-written templates does not scale across languages and accents the way a model can.

Amazon DynamoDB is doing the memory. The cross-session consistency check, comparing this call against what a genuine caller has sounded like before, needs fast lookups of voice fingerprints tied to a caller’s history. This is a live, low-latency lookup on every call, not a batch job, which is exactly the shape DynamoDB is built for.

Take away real-time streaming or the memory layer and you no longer have a live trust layer, you have an offline audio classifier, which is a different and much weaker product.

The AWS stack we will use

What	What it does for us	How important
Amazon Chime SDK	Captures and streams live call audio in real time, the backbone that makes this “live” instead of “after the fact”	Essential. Nothing else gives us real-time voice streaming out of the box
Amazon Bedrock	Turns a raw risk score and detection signals into a plain-language reason and recommended action	Essential. This is what makes the output usable by a non-technical frontline staff member
SageMaker or Lambda	Hosts the acoustic, prosody, and consistency detection models and runs inference on each audio chunk	Important. Where the actual detection math happens
DynamoDB	Stores caller voice fingerprints and session history for the consistency checks	Important. Makes the system get smarter about known callers over time

What	What it does for us	How important
API Gateway + Lambda	Exposes the REST interface so banks and enterprises can integrate this into their own call systems	Important. This is the actual product surface for a real customer
EventBridge	Watches for risk scores crossing a threshold and triggers alerts and escalation workflows	Useful. Keeps the alerting logic decoupled and configurable
Amplify Hosting	Serves the live dashboard, the risk meter, alert history, and escalation controls	Useful. What the judges and any real user actually look at

We are deliberately building this on the cloud stack, not the local one. The whole premise is a live service that a bank or telecom operator plugs into, not a tool that runs on one person's laptop. That is a Ship It project honestly, not a workaround.

Handling many languages and accents

The problem statement is explicit that this has to work across Indian languages and regional accents, not just clean English speech. So the detection layer should not depend on understanding what is being said, only on how it is being said, meaning the acoustic and prosody checks work the same way regardless of language. Language-specific tuning can come later. For the hackathon, we prove the pipeline works end to end on a couple of languages and accents, not all of them.

What we will show in the demo

One continuous run, not a feature tour. It goes like this.

A call comes in on screen. The risk meter sits low while a genuine voice speaks normally. Partway through, the audio switches to a cloned version of the same voice, made from a short public sample, the same way an attacker would do it. The risk meter climbs live, in real time, while the call is still playing. A plain-language flag appears: what was detected, and why. A recommended action shows up: verify with a callback before approving anything. The whole sequence, genuine voice to detection to recommendation, happens in front of the judges without a single cut.

That one run proves everything at once. It is live, it explains itself, and it tells a human what to do next.

Who does what

Four people, four lanes.

- 1. Streaming and pipeline.** Amazon Chime SDK integration, audio chunking, getting live audio flowing into the detection models with low enough latency to feel real time. This is the hardest part and starts on day one.
- 2. Detection models.** Acoustic and spectral analysis, prosody and rhythm analysis, cross-session consistency checks, and combining all three into one running risk score.

3. Explanation and alerting. Bedrock integration for turning scores into plain-language reasons, EventBridge threshold logic, DynamoDB for caller history, the API surface.

4. Frontend and story. The live dashboard with the risk meter, the demo script and recording, the blog post.

One thing to sort out first

Get a single audio clip streaming end to end through Chime SDK into a placeholder model that just returns a random score, before anyone writes a single line of real detection logic. A live pipeline that always says “50% risk” is more valuable on day one than a brilliant offline detector that only works on a saved file, because the entire demo depends on the live path working, not on the model being perfect. Once audio is flowing and a number is moving on screen, everything else is refinement.